Module 1: Introduction to Machine Learning

Video Transcripts

Video 1: Introduction to Machine Learning

"If you wish to converse with me, begin by defining your terms." This was a model of Voltaire's, and it is still good advice. So, let's begin by defining the concepts in the title of this course, artificial intelligence and machine learning. Well, the truth is that there is no consensus on the precise meaning, but we'll give a sense of how they are typically used. Artificial intelligence is the older of the two concepts dating back to the mid-20th century when the early pioneers of computing thought deeply about whether a machine could be 'intelligent.' But what do we mean by intelligence? This is a difficult thing to define. Even if we focus on human intelligence, we find different types: musical intelligence, verbal intelligence, visual spatial intelligence, social intelligence, and so on. And our definition must capture all of these.

How about this? An intelligent agent, whether human, machine or other, is one that can find efficient solutions to problems by consulting a model of the world that is learned from interactions with the environment. The agent is not just trying things at random. Through experience, it has built a detailed model of how to approach the problem at hand. Notice some keywords: efficient solutions to problems. This could be the problem of how to play chess in an advanced level, a hard problem for most humans, or how to recognize whether a photo contains a face, an easy problem for most sighted humans. The important thing is that the problem should be a task for which you can quantify your performance. Either in a discrete binary sense: I win, I lose, I got the right or wrong answer. Or in a more continuous sense: I got 87 out of 100 on the test. And the agent should find efficient solutions. It is not good enough to search endlessly through all possibilities, giving equal weight to good and bad ideas.

And what is the model? In order for the agent to devise an answer to the problem, it must be able to estimate the quality of alternative answers. To do this, it uses a model, which is in our case a mathematical representation of the problem. And finally, the model is learned by

the agent through interactions with the environment. And this is where machine learning and data science enter the picture. Whenever the agent interacts with the environment, the environment responds with some data, which the agent uses to update and improve its model. Machine learning is the study of all of the different ways in which we can build models from data. The models might simply distinguish different types of objects. For example, this robot is noticing that even though all individual fruits are different, it still makes sense to think of a whole class of fruit called apples, and another class of fruit called oranges. In its quest to understand fruit, it may then try to create a list of features that characterize 'apples' and distinguish them from 'oranges.' Characteristics such as their color, their smoothness, their water content, etc.

With this model in hand, the agent can then solve problems, like sorting a basket of fruit. Machine learning offers algorithms for processing data that construct models of a wide variety of phenomena. The more complex the task, the more complicated the model needs to be to represent it. The more data we will need to train the model, and the more computational power we will need to process that data.

To get a better grasp of the machine learning task, let's consider a game in which you are presented with a mystery box. The box has two slots, but you cannot peek inside, and you have no clue about its inner workings. The two slots are labeled input and output. And when you place a list of numbers into the input, the box produces a single number from the output. Your task is to build a mathematical model that you can use to predict the output for any given input. So, let's introduce some notation. We use the bold letter X for the input. It is bold to indicate that it is a list of numbers. We may also refer to this list as a vector, an array, or a tuple. They all mean roughly the same thing. The output of the box is y. This is just a single number, a scalar. So, we do not use a bold font. Our task is to construct a model, which is a function, f, that takes inputs and produces the predicted output. The model f will be good, in so far as the predicted output $\hat{y}$ matches the actual output y.

So how should we proceed to construct f? Well, the reasonable thing to do first is to collect some data. That is, let's enter a bunch of numbers into the mystery box and record the resulting output into a table. Each row in the table is a single trial or sample. The row

records all of the input and the observed output. In this case, we did eight trials, numbered 0 through 7 on the left. Each input vector has five entries, x1 through x5, and the rightmost column records the resulting output. This is our data in table form. With this table, we can now start looking for patterns.

Suppose we suspect that the fifth input, x5, may have a stronger influence on the output than all other inputs, x1 through x4. Then we would make a plot of x5 versus the output and obtain the scatter plot. It does in fact seem like there is a relationship. The output tends to be larger when x5 is larger. This trend suggests an approximate model. The output is a straight-line function of x5, and we can ignore all other inputs. In our model, they have no effect. This is not a perfect model. You can see that it makes errors. So, this is just one possibility of the many models we could choose. In later lectures, we will study a variety of models that apply in different contexts: linear regression models, such as this one, linear classifiers, support vector machines, and neural networks.

The first question you should be asking is how we will determine whether our prediction model is any good. Say the output y is 2.0 and the model guesses $\hat{y}$ equals 2.01. Is this as wrong as guessing $\hat{y}$ equals 201? Is there any partial credit in this game? Fair enough. You are also given a function that will produce a score for each of your predictions by comparing the predicted value $\hat{y}$ to the actual value y. But the score is actually a penalty, and your objective is to get as small a score as possible. The function that produces the score is called the loss function, and we denote it with the letter L.

Here's a common example: L takes in the actual result and our predicted result and scores it as the square of the difference. You get zero penalty if you precisely predict y and your penalty increases quadratically with the error. There are other possible loss functions we can use, and we can manipulate the loss function to our advantage. And we will see that later in the course. As you are gathering data, you notice something interesting. The mystery box is non-deterministic. That is, you enter the same input twice and get different outputs. This means there is no hope of achieving a perfect score of zero penalty on the task because no matter how good your model is, there is simply not enough information in the input to precisely predict the output. There is a lower bound on the score that you can

achieve, and this lower bound accounts for all of the natural internal uncertainty of the box. This is the best you can hope for. From this observation, you've realized that the tools of probability and statistics will have a role to play in building your model. We will spend the first few lectures on the tools of probability and statistics that are needed for machine learning.

The mystery box thought experiment may seem fanciful, but it actually captures the essence of machine learning, and it will apply to all of the techniques that we will study in this course. Let's bring the analogy closer to the real world. The mystery box is actually some system that you wish to study. Often the system is the human brain, or a population of people, or other highly complex systems. It is not literally hidden from us. We can see the human brain, and in fact neurologists are learning more every day about how the brain works at a neuronal level. However, we still cannot use this knowledge to, for example, predict that a person will say the phrase "letter A" when seeing a particular scribble on a piece of paper. The human brain is too complex for that and it makes more sense to use the non-mechanistic data-centric techniques of machine learning for this task.

On the other hand, simple systems are often easier to model with mechanistic techniques. Imagine that the problem is to predict the time it takes for an apple dropped from a building to reach the ground, given the height of the building is input. In this case, the system is sufficiently simple that we can model it using the known acceleration of gravity and the properties of air. We need very little data or no data at all to produce a good model. This does not mean that our model will be perfect. There are still unknowns, such as the wind speeds. Part of the art of machine learning is to know when tools should be applied. Throughout the course, we will see many applications of machine learning to real world problems, and this will help us to build a sense for when data-centric techniques are appropriate.

Video 2: Distribution and Random Variables

Let's start by considering a system with no inputs. And every time we sample the system we obtain a different number. Here's a list of the first 20 numbers we sampled from this

mystery box. How should we model the fact that the system is non-deterministic? That is, that it produces different outputs for the same inputs or even for no inputs at all.

This is where probability theory comes in. Our model of the system will consist of a probability density function, or a PDF, also known as a distribution. Our goal will be to infer something about the distribution from the observed data. There are two types of distributions, depending on whether the output is discrete valued or continuous valued. The toss of a coin is an example of a system with a discrete valued output. There are only two possibilities: heads or tails. The height of people in the population is an example of a continuous value distribution. The output number can be any real number, 160cm, 161cm, or anything in between. For example, 160.3cm.

A distribution, or probability density function, is used to compute the probability of every possible value of the output. For discrete distributions, we read the probability of each value directly. For example, here the probability of heads is 0.4 and the probability of tails is 0.6. This is clearly not a fair coin. However, for continuous value distributions, it does not make sense to ask, for example, for the probability that a person is 160cm tall because no one is exactly 160.0000cm tall. What makes sense is to ask for the probability of a range of heights. For example, from 160 to 165cm. The probability of heights from A to B centimeters corresponds to that area under the curve between the two values. As you know from calculus, the area under a function is the integral of that function. Notice that we denote the probability of something with a capital (P), and the probability density function with a lowercase (p). So the probability of heights falling in the range A to B equals the integral of the probability density function from A to B.

Variables, such as Y, whose values change each time you sample them, are known as random variables. It is customary to denote random variables with capital letters. So from now on we will use the capital letter "Y" for the generic output of the system and lowercase "y" for samples of the output. The tilde symbol means that a random variable is distributed according to, or sampled from, a particular distribution. So here we are saying that the random variable Y is distributed according to the distribution Py. Square brackets

surrounding a lowercase letter denote a data set of size n. In this case, n equals ten data points sampled from the random variable Y.

Video 3: Expected Value and Variance

We have seen that the output of a system is represented as a random variable, which is a variable whose value is sampled from a probability density function or distribution. So let's take a deeper look at distributions. Not all functions are probability density functions. For a function to qualify as a distribution, it must satisfy these two criteria. First, it can never be negative. All of its values must be greater or equal to zero. This is reasonable because it does not make sense to speak of negative probabilities. Probabilities are always between zero and one. Does this mean that the distribution can never go above one? In the case of discrete distributions, it does. But not for continuous distributions. In continuous distributions, you may see values go above one. However, the area under the curve between any two vertical lines can never exceed one.

The second requirement for a distribution is that the sum of all of the values of the distribution must be equal to one. In this example, we can verify that the sum of 0.05 plus 0.1, plus 0.2, 0.3, 0.15, 0.1, and 0.1, is indeed 1. So, this function is a discrete distribution. For continuous distributions, the requirement is that the total area under the curve, or the integral from minus infinity to infinity, equals one. Let's assume that the function P_y qualifies as a distribution. We can now think of the random variable Y as sampled or distributed according to P_y.

Next, we will look at two important properties of random variable Y: its expected value and its variance. First, the expected value is indicated like this, with a capital letter E, followed by the random variable in square brackets. You can also call it the expectation or the mean. And you can say expected value expectation or mean of the random variable Y, or of the distribution P_y. They're all the same thing. You will often see the expected value of Y written as μ_y. So, you should be aware of this notation.

Expectation is defined mathematically as the sum, or the integral in the continuous case, of the probabilities P_y multiplied by the values Y. The mean or expectation of a distribution

can be understood as its center of gravity. That is, if we build the shape of the distribution by stacking boxes on top of each other, and then we find the balance point where the moments generated by those boxes cancel out, then this point coincides with the expected value of the distribution. We can balance the distribution on a fulcrum placed at the mean, but contrast the mean with another type of average, the median. The median is obtained by finding a vertical line that separates the boxes into two equal parts, with the same number of boxes on either side.

Imagine now that we moved two of the boxes from the rightmost stack much further to the right. This would have an effect on the balance point, moving it somewhat to the right. However, it would have no effect on the median, which would remain at 1.0 because there are still the same number of boxes on either side of the line. So, the mean is more sensitive to outliers, such as these two boxes, than the median.

Now, let's do a quick example of computing the expected value for a continuous distribution. Let's say we have a random variable that can take any value between zero and two, all with equal probability. This is the formula for this distribution, and it equals one half between zero and two, and zero everywhere else. You can verify that the area under the curve is one. We compute the expectation by applying the formula; E of Y equals the integral from 0 to 2 of Y times the distribution P_y. Substituting 0.5 for P_y, we get the second line. We can then take 0.5 out of the integral and apply the antiderivative of Y, which is Y squared over 2. Evaluating this between zero and two gives us four over two. And 0.5 times 4 over 2 is 1. So, we have found that the expected value of this distribution is one, which is the midpoint between zero and two. This is reasonable since the distribution is symmetric, and so its balance point should be at the middle.

If the expected value of a distribution represents its center, then the variance is a measure of its width. Just as you will see two notations for the expected value, an E or a mu, you will also see the variance represented with the letters, Var and square brackets, or as a sigma squared. Also, one can speak of the variance of the random variable Y, or of the distribution. These again are the same thing.

The variance is defined mathematically as the expected value of the square deviation of Y from its mean. A distribution with large variance is wider and shorter, generally speaking, than a distribution with small variants. So the variance gives us a sense of the uncertainty in sampling a random variable. Small variance means that we are more likely to sample values that are closer to the mean, whereas large variance means that we are very uncertain about the values that we will obtain. Notice that the definition involves squaring the random variable. So the units of variants are the square of the units of Y. If Y is in meters, then the variance of Y is in meters squared. For this reason, it is common to work with the standard deviation of a random variable rather than the variance. The standard deviation is simply the square root of the variance, and it is denoted with a sigma.

Why then, if it is less robust, do we focus on the mean instead of the median? The answer to this question is given by a very important property known as the law of large numbers. Consider now an arbitrary distribution. It could be continuous or discrete, it doesn't matter. And compute its mean according to the mathematical definition. Now, sample the distribution n times, and put the result into a list, Yn. We can take the average of the list by adding up the values and dividing by n. Now, repeat that with n is equal to 11, then n is equal to 12, 13, and so on. The law of large numbers says that the average will converge to the expectation of Y in the limit as n becomes large. This plot shows that the law of large numbers in action. The blue line is the sample average for a sample of size n, plotted against n on a semi-log scale. So n varies from 10 to 1 million. We see that this average gets closer and closer to the expected value of Y, which in this case is one, as n grows. This gives us a useful interpretation of the expected value. It is the average over a large number of samples. For example, if Y represents the number of servings of fruit a person eats in a week, then the expected value of Y is the weekly fruit consumption averaged over many weeks. Perhaps over the person's lifetime.

Video 4: Introduction to pandas

In this video, we will begin to familiarize ourselves with the pandas package. Pandas adds functionality to Python for operating on tabular data and we have already seen that tables are a common format for datasets in machine learning applications in which each row is a

sample from a multivariate distribution and each column is a feature or a single random variable being sampled.

So, why do we need pandas? Okay, you may be familiar with other types of lists available in Python. There are Python native lists like this one and NumPy arrays like this one down here. And the difference is that Python native lists are heterogeneous, meaning that they can hold different data types. This native list has numbers 0.1 and 0.2 and strings a and b, whereas this NumPy array is homogeneous. NumPy arrays are all homogeneous. This one has only numbers and that homogeneity of NumPy arrays is what makes it very efficient for numerical computation and that's why it is preferred for all sorts of numerical and machine learning applications.

Now, pandas takes this one step further and offers a dataframe type, which is exactly what we need for tabular data. This is a dataframe and we see that it has three columns —X1, X2, and Y— and each column is homogeneous in itself. X1 has only numbers, X2 has these strings, and Y is only numbers. So, there is a lot to learn about pandas and we've included here a link to the documentation of pandas and also to the user documentation and these are very useful documents and I encourage you to have a look at them to learn more deeply about pandas. We will just scratch the surface here and we'll be seeing more, as we progress in the course.

Like many packages in Python, pandas has a standard alias, which is pd. So, when we import pandas, we will always refer to it as pd. So, let's run this line here and now we have imported pandas. The first thing that you will do when using pandas is to create a dataframe. That is, we'll create one of these tables and we can do that either from data that is stored in memory on our computer or by importing data from an external source.

So, let's try to do it from data that is currently in memory. And in this case, we'll do it from a dictionary. This dictionary has its keys, a string A and a string B, and these will serve as the headers for our dataframe or table and the values in the columns will be the lists that are referred to by the keys. So, we have now loaded this dictionary into memory and the way we now create the dataframe is by calling the dataframe constructor and passing in that dictionary and we assign that to some variable, let's call it X, and now we can execute this.

And here we have the dataframe X with the two columns, A, and the A column has the values that we had assigned to it and B is all strings. Okay, but I see here that there's a third column, this one on the left that has a 0, 1, and 2. This is called the index of the dataframe. And every dataframe must have an index because we did not explicitly tell pandas what to use as its index, it created one by default and the default index is integer IDs of the rows starting from zero. So, that's what this is and you know it's the index because it doesn't have a title here or rather the title is not at the same level. We'll see an index with the title in a little bit.

Okay, so we can change the index, we can use our own index, and there are three ways to do this. We can either pass an index to the constructor with an index argument or we can use the set index method to assign a column to be the index or we can keep the default index, which is what we have up here. Okay, so let's try that. First, we will try passing an index input argument. So, let's take this line, put it here, and I'm going to pass in an argument called index and I'm going to assign it, say "row0", "row1" and "row2." And let's look at that. So, now you see that the index is row0, row1, and row2. We've explicitly given it an index that we want and now we can refer to those rows with the names that we've chosen. Another method, if we've already created a dataframe without an index, this one again, is we can assign one of the columns to be the index with the set index function. And what this will do is it will create a new dataframe with that column as the index. So, now we see that the column A, with its values 25, 56, and 93, are the index of this dataframe.

Okay, well that was just a little aside on the index, which is an important part of dataframes, but we were talking about how to create a dataframe. So, let's go back to that topic and see how we can create a dataframe from an external source, not from internal memory. And pandas provides many read functions that we can list out by typing "pd.read_" pressing tab, and this activates the autocomplete function of Jupyter, which lists all of the different read functions we have available. And I see here that we can read from .csv files, from Excel, and from many other data sources that I don't even know. So, you have all of these possibilities. We in this course will mostly be reading from Excel, I mean from .csv files. So, let's do that. To do that, I'm going to load this file called "celebrity heights" and you can find this file in the data folder. And what I say is "read_csv" and I pass in the file name. And that will create

a dataframe from the data that is contained in that file. Okay, here we have it. That file contains the ID, first name, middle name, last name, full name, and heights in feet, inches, and meters of 5,502 different celebrities. Well, and I did not provide an index, so it has attached an integer index.

Let's now create a new dataframe with the full name as index. We've seen how to do that and now this dataframe has an index, which it has taken the full name column and put it into the index. And now we don't have a full name column in CH2. Now, remember for later that CH is the dataframe with integer index and CH2 is the dataframe with string index. The next thing we're going to do is load our data, not from data that we have in the memory on our computer, but from an online source. So, we will load data on educational attainment and personal income from the California Open Data portal. Let's open that. And here we see the URL that provides this data. I'm going to copy that URL and paste it in here and we're going to use the same pd.read_csv function, but now we pass in the URL. And now pandas has gone to that resource, downloaded it, and put it into a dataframe. So, there we have it. Okay, great.

Once we have the dataframe, usually the first thing that we want to do is just get some summary of the data that's in there. So, there are three different methods of the pandas dataframe that you should have at your fingertips. The first one is called info and this just gives us a quick summary of the data types contained in the dataframe and the columns. So, this one on educational attainment has six columns. The first four, this object here, tells us that they're strings. So, even say the Age in this case is a string. That may be something you might want to deal with. And the last one, Population Count, is a number of float. Okay, that's interesting.

The next thing we want to do is called the describe method, which gives us some summary statistics on the numerical data. So, because this table only has one numerical column, then we will only get one column and it tells us how many data points there are, the mean of the numbers, the standard deviation, and the various quartiles. And another two functions that are very often used are the head, which just gives us the top bit of the dataframe, and tail

gives us the lower bits, so the last few rows. And you can pass into these, say a "10" will give us the last 10 rows or the first 10 rows.

Video 5: Data Selection in pandas

Now that we have learned how to create a dataframe, let's see how we can select data from the dataframe. There are three basic ways to select data from a dataframe. Either with bracket selection, with label-based selection, or with integer-based selection. And you can see here the differences in syntax between these three. We're going to go through all three of them.

First, bracket selection. The syntax for bracket selection is to write the name of the dataframe, followed by a bracket and a selector inside. And we've already seen me do that a couple of times. To select the columns, the selector is going to be a column label or a list of column labels. And to select rows, the selector will either be a slice or a Boolean mask. So, we can use brackets selection to either select columns or rows, but not both at the same time. Let's try these different things.

We have in memory now our celebrity heights table and also the index-based one and also the name-based one. So, let's select from the index-based one, the column of full names. There you have it. Or we could select multiple columns by giving a list of strings. So, we can say full name. And let's see what other columns we have here. Last name. Okay, so we supply a list of strings and it gives us a new table with those columns selected. So, that is multiple columns selection with bracket indexing.

We can also select rows, as I said, with slice indexing. And you may already be familiar with slices in Python. These are arrays of indexes that are created with colon. So, if I want, say the first four rows of the table, I use the slice that gives us the first four rows of the table. Now we can also select rows with a Boolean mask. So, let's say for example, we wanted to find who are the celebrities whose height in meters is, say, greater than two. Okay, if I do this, this will return a list of Booleans false for those celebrities that are shorter than two meters and true for the ones that are taller than two meters. This is called a Boolean mask. And we can pass this Boolean mask now into bracket selection and it will return the full

table for all of those celebrities that are over two meters. Okay, so that is selecting with some Boolean criterion and it's very useful for for filtering rows. Okay, let's shorten that by saying 2.2, for example. So, there are only six celebrities that are over 2.2 meters. That's bracket selection.

Let's move on to loc selection or label-based selection. Loc is a more general and powerful way of selecting from a dataframe because it allows you to select both rows and columns. So, this will give us much more interesting options for selecting data. The row selector can be an index or a list of indexes in addition to the slice and Boolean masks that we were able to use with bracket-based selection. So, now we see the indexes come into their own with loc. And you can also have a column selector which can be as with the bracket-based, a string label or a list of string labels. So, you should think of loc as more general than bracket and it can do almost everything that bracket-based can do, except with one caveat. You cannot run loc of a slice when the index of the dataframe is not integer-based. So, make sure that if you're running loc of a slice that your table has an integer-based index.

Let's look at some examples. So, we have, you'll recall, the celebrity heights table. And we can select rows from here, just as with bracket selection, with a slice. Or we can give it an index or we can give it a list of indexes: 0, 5, and 7. Right. And if we're looking at our table with full name string-based index, we can pass one of those in to loc to select it. And we could even pass in a string of those. Okay, and obtain a subselection of the table in that way. Now, the most interesting thing is that we can select rows and columns simultaneously with loc. So, let's say we have this selection and we only want to keep the middle name and last name from there. So, we pass in a column selector. And that will produce a new table with only the middle name and the last name.

Okay, with loc we can also do the Boolean masking, as we did before. So, we have up here an example of Boolean masking and we can run loc on that and it's the same thing, but as I said before, we can also pass in a column selector to loc, and it will do the right thing. Okay, so loc is a much more powerful and general approach to selecting than brackets. So, think of brackets as a syntactic sugar for loc.

Finally, we have iloc and iloc is purely integer-based. So, usually it is used when our dataframe has an index and column headers that are just integers. So, these are numerical dataframes. And then we can use iloc and the row and column selector will just be integers or a slice. Okay, so it will work for actually any type of dataframe. So, let's pass in here. Let's say we want the first three rows and the first three columns of this dataframe, we just use iloc. And so iloc only takes slices or integers. Let's say we wanted the 0th, the 10th, and the 20th row, we can use iloc.

Okay, so to summarize, we have seen the three methods for selecting data from the dataframe. And with practice, you will become familiar with all three of them.

Video 6: pandas Operations and Plots

We've learned how to create dataframes in pandas and also how to select data from those dataframes. And now, we're going to see how to do simple operations on columns of the dataframe.

So, we'll begin by importing NumPy in pandas, as we usually do, and we'll create a very simple table and look at it. This table has two columns, A and B, and three rows. And all of the data here is numerical. Now, if we look at the type of the columns. So, let's take the first column and see what type it is. It's an object of type series. A series is, apart from the dataframe, the other user-facing object class in pandas. And here, I've added a link to the documentation on series. And if you go down, you'll see that there are many interesting and useful methods attached to the series object. Here, we will look at just a few of them: The sum, multiply, and add. And you can check out the others in the documentation.

So, these operations are similar to what you would do in say, a spreadsheet, like Excel. If you wanted to, say, get the summation of the values in a column, multiply a column, or add two columns. So, let's find the summation of a column. Take the column A and just run the sum function on it. And we get that 0.1 + 0.2 + 0.3 is 0.6. And we could do that also with column B. If we wanted to add columns A and B, we would use the add functions. So, table A, table column A. We're going to add to it. And we're going to pass into it what we want to add to that column. We're going to add column B. And let's say we want to assign that to

column C. That is what we would run. And then look at it. Now we have a column C that has the sum of columns A and B. We could do the same thing with multiplying two columns. So, all I do is I say multiply, and I'm going to store that in column D. There we have column D is the product of columns A and B.

So, as I said, there are many other things that you can do with the series object, which represents a column in a pandas dataframe. Let's move on to plotting. Later on in the course, we'll see another package for plotting called Seaborn. But for now, I want to show you the simplest way to plot data from a dataframe. And that is to use the plot function. Here, we have the documentation for the plot function. And we see that when we create a plot, we pass in this argument called kind. And that tells the plot function what type of plot to create. It can be a line plot, a bar plot, horizontal bar plot, histogram, etc. Let's try it out. First, we will load some data from scikit-learn datasets. So, scikit-learn offers several test datasets for playing with different methods, like regression. And the Iris dataset is a classical dataset for regression. This is the code for loading the Iris dataset. And let's have a look. So, I've put it into a dataframe. And what this dataset contains is measurements from 150 different Iris flowers and measurements of their sepal length, sepal width, petal length, petal width. And on the right is the target variable, which is the variety of Iris flower that it is. And so, this dataset is used to create regression models that will predict what type of flower corresponds to some measurements of the petal measurements.

So, let's create some plots of this data. First, we're going to create a histogram. And what I'd do is I call the plot method and I pass kind='hist'. What this will do is it will create histograms of all of the columns, including the target. And maybe this is not what I want. I only want a few of the columns. Let's say, sepal length and sepal width. And so I passed that in as the y argument. And let's put quotes around these. And now we have histograms only of the sepal length and sepal width.

Another type of plot that we might want to create is a scatter plot. Okay, so let's do that. "df.plot" and we tell it that the kind is going to be a scatter plot. And when we do a scatter plot, we need to tell it what variable to put along the x-axis and what variable to put along the y-axis. So, let's just use these two that we have already here. These are the names of the

columns that it's going to use for each of the axes. And I run that and I get a scatter plot. Now, I can pass in other arguments to this plot function. Such as, if I wanted to add a grid, I'd pass "grid=True." And you can find other arguments that are helpful in the documentation.

Another type of plot that we might want to make is a line plot. Now, line plots are more common for time series data. So, I'm going to load here another dataframe or a dataset, called the timeseries dataset, and it's in a .csv file. Here is the dataset. It has 1,096 measurements, taken over three years of power consumption. A Wind energy production and Solar energy production and the sum of the two. So, if we wanted to make a plot and we're going to make it a line plot. And we're going to say that along the x-axis, we're going to put the Date. And along the y-axis, let's say we will have the level of Consumption. So, that is the plot of Consumption.

We might also want to have more than one line here. And so, we could say, turn this into a vector. And say we want Wind and Solar. And that's what that looks like. Again, we can add a grid. And one other thing we can do is we can change the size of the figure. Let's say, make it a little bit bigger. Okay, by passing a figsize argument.

So, what if we want to create a summary of the total Consumption Wind power and Solar power over that period, and we wanted to report that as a bar plot? This is how we'd do it. So first, let's look at our table. We'd want to extract the Consumption, Wind, and Solar columns. So, let's do that with bracket selection. And that's what that looks like. Just those three columns. Then take the sum of those columns. And here, we have the total Consumption and production of Wind and Solar. And now we can make a bar plot. So, we make a plot, and we pass, the kind is going to be bar. And there you have it. We have a plot of the total Consumption, the total production of Wind, and the total production of Solar energy.